

Manual de Estándares de Programación

1. Introducción

1.1 Propósito

Este manual establece los estándares y prácticas recomendadas para el desarrollo de software en Nero. Su propósito es asegurar la consistencia y calidad del código.

1.2 Ámbito

Este manual se aplica a todos los proyectos de desarrollo de software realizados en Django.

2. Convenciones de Nombres

2.1 Nombres de Variables

- Utilizar camelCase para variables y escribirse en ingles
- Ejemplo: userName.

2.2 Nombres de Funciones/Métodos

- Utilizar camelCase para nombres de funciones y métodos.
- Los nombres deben ser descriptivos y expresar claramente la acción realizada.
- Ejemplo: calculateTotalAmount(), sendNotification().

2.3 Nombres de Clases

- Utilizar PascalCase para nombres de clases.
- Ejemplo: UserProfile, OrderProcessor.

2.4 Nombres de Archivos

- Utilizar snake_case para nombres de archivos.
- Ejemplo: user_service.py, order_processor.py.

3. Estilo de Código

3.1 Indentación

- Utilizar 4 espacios para la indentación.
- No utilizar tabulaciones.

3.2 Espaciado

- Utilizar un espacio después de los operadores (+, -, =, etc.).

Ejemplo Correcto: $x = y + 1$

Ejemplo Incorrecto: $x=y+1$

- No colocar espacios antes de los paréntesis de funciones y llamadas.
- Ejemplo: `if (condition) { y x = y + 1.`

3.3 Longitud de Línea

- La longitud máxima de una línea debe ser de 100 caracteres.
- Dividir líneas largas en múltiples líneas según sea necesario.

3.4 Comentarios

- Utilizar comentarios en línea para explicar partes complejas del código.
- Utilizar comentarios de bloque para describir la funcionalidad de métodos y clases.
- Utilizar docstrings para documentación de métodos y clases.
- Ejemplo:

```
def calculate_total_amount(items):
```

```
    """
```

```
    Calcula el monto total de una lista de ítems.
```

```
    :param items: Lista de ítems.
```

```
    :return: Monto total.
```

```
    """
```

```
    # Código aquí
```

4. Estructura del Código

4.1 Organización de Archivos y Directorios

- Colocar archivos de código fuente en el directorio `src`.
- Colocar archivos de prueba en el directorio `tests`.

4.2 Estructura de Funciones y Métodos

- Cada función o método debe realizar una sola tarea.
- Limitar la longitud de las funciones a un máximo de 50 líneas.
- Utilizar nombres descriptivos para funciones y métodos.

5. Manejo de Errores

5.1 Excepciones

- Utilizar bloques try...except para manejar excepciones.
- Especificar excepciones concretas en lugar de utilizar except genérico.
- Ejemplo:

python

Copy code

try:

```
# Código que puede lanzar una excepción
```

```
except ValueError as e:
```

```
# Manejo de la excepción
```

5.2 Mensajes de Error

- Los mensajes de error deben ser claros y descriptivos.
- Evitar mensajes genéricos como "Error desconocido".

6. Prácticas de Seguridad

6.1 Validación de Entrada

- Validar y sanitizar todas las entradas del usuario.
- Utilizar técnicas de prevención de inyecciones SQL y XSS.

6.2 Protección de Datos

- No almacenar datos sensibles en texto claro.
- Utilizar cifrado para datos sensibles.

6.3 Autenticación y Autorización

- Implementar autenticación segura y gestión de sesiones.
- Definir claramente los roles y permisos de usuario.

7. Uso de Herramientas y Bibliotecas

7.1 Linter y Formateadores

- Utilizar [Por escoger (tentativo flake8)] para el análisis estático del código.
- Utilizar [Por decidir ()] para el formateo automático del código.

7.2 Bibliotecas y Frameworks

- Utilizar bibliotecas y frameworks aprobados por el equipo.
- Evitar dependencias innecesarias y no mantenidas.

- Agregar siempre a documentación y requerimientos

8. Control de Versiones

8.1 Commit Messages

- Utilizar mensajes de commit claros y descriptivos.
- Seguir el formato: [Tipo] Descripción (Ejemplo: Fix: Corrección de error en la validación).

Tipos Comunes:

- **feat**: Nuevas características o funcionalidades.
- **fix**: Corrección de errores.
- **docs**: Cambios en la documentación.
- **style**: Cambios que no afectan la funcionalidad del código, como el formato.
- **refactor**: Cambios en el código que no añaden funcionalidades ni corrigen errores, pero que mejoran la estructura.
- **test**: Añadir o modificar pruebas.
- **chore**: Tareas generales y mantenimiento del proyecto.

8.2 Branching Strategy

- Utilizar una estrategia de ramas basada en main, develop, y ramas de características.
- Crear ramas para cada nueva característica o corrección.

8.3 Revisión de Código

- Realizar revisiones de código para todos los cambios importantes.
- Proveer retroalimentación constructiva y específica.

9. Documentación

9.1 Comentarios en el Código

- Incluir comentarios explicativos donde sea necesario.
- Mantener los comentarios actualizados con respecto al código.

9.2 Documentación de API

- Utilizar herramientas como Swagger para documentar las APIs. (sugerencia por revisar)
- Incluir ejemplos de uso y descripciones detalladas de los endpoints.

10. Pruebas

10.1 Estilo de Pruebas

- Escribir pruebas unitarias para cada módulo.
- Escribir pruebas de integración para flujos de trabajo importantes.

10.2 Cobertura de Pruebas

- Asegurar una cobertura de pruebas mínima del 80%.
- Utilizar herramientas de cobertura para evaluar y mejorar la cobertura.

11. Ejemplos y Plantillas

11.1 Ejemplos de Código

- Proporcionar ejemplos de código que sigan los estándares establecidos.
- Incluir ejemplos para las prácticas comunes en el proyecto.

11.2 Plantillas

- Proporcionar plantillas para la documentación, mensajes de commit, etc.
- Ejemplo: Plantilla para mensajes de commit.